

INSTITUTO FEDERAL

Mato Grosso do Sul

Prog. Disp. Móveis II

Milena Alegre

Recurso Nativo x Cross-Platform

MÓDULO 1: FILOSOFIA NATIVA

O Dilema — Google Maps vs. Apple Maps



Consumo de Recursos: O SDK do Google Maps no iOS roda camadas extras de compatibilidade, aumentando o footprint de memória.



Sinergia do Sistema: O Apple Maps integra-se nativamente ao CarPlay, Dynamic Island, Apple Watch e Atalhos da Siri sem overhead.



Veredito de Engenharia: Utilizar o mapa nativo correto evita sobrecarga de renderização e otimiza radicalmente a UX.

Recurso Nativo x Cross-Platform

MÓDULO 1: FILOSOFIA NATIVA

Por que a Experiência Nativa Vence?



Acesso ao Hardware

Chamadas nativas acessam diretamente os drivers de GPU e sensores sem pontes intermediárias.



Look & Feel Original

Respeito absoluto aos padrões visuais e respostas táteis que o usuário já domina.



Gestão Térmica

Menos processamento secundário resulta em menor aquecimento e maior estabilidade do aparelho.



Confiabilidade

Independência de bibliotecas de terceiros diminui o risco de quebras em atualizações do SO.

Recurso Nativo x Cross-Platform

MÓDULO 1: FILOSOFIA NATIVA

A Anatomia do Hardware e SO

Camada de Abstração

Fluxo: App → Framework (Swift/Kotlin) → APIs do SO → Kernel → Hardware.

Custo de Abstração: Adicionar "bridges" cross-platform inclui latência e overhead na troca contínua de dados.

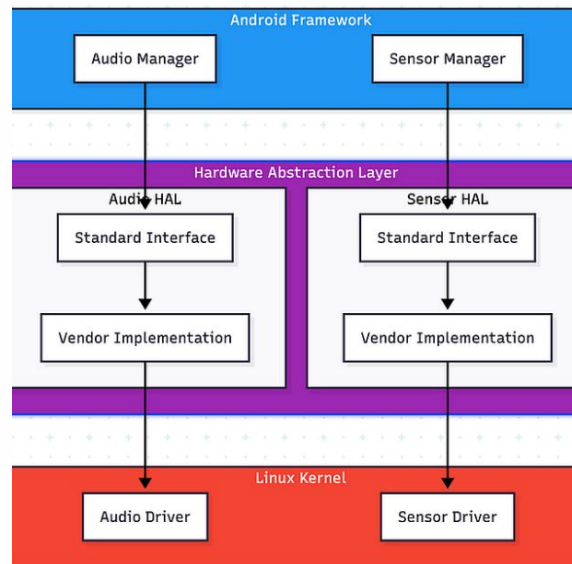
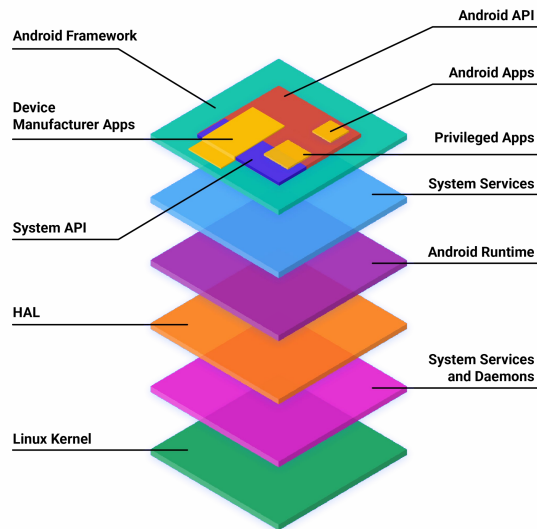
Acesso Direto: APIs nativas eliminam a necessidade de serialização JSON ou conversões pesadas em tempo de execução.

**"sobrecarga" ou "custo adicional"*

Recurso Nativo x Cross-Platform

MÓDULO 1: FILOSOFIA NATIVA

A Anatomia do Hardware e SO



MÓDULO 1: FILOSOFIA NATIVA

Consumo de Energia & Ciclo de Vida



Gestão de Energia

APIs nativas acessam registradores e coprocessadores de baixo consumo (como sensores de movimento dedicados).



Lifecycle Rígido

O SO encerra ou congela processos com maior agressividade se detecta alto consumo indevido de memória.



Impacto em Background

Execução em segundo plano sem APIs nativas adequadas é sumarissimamente bloqueada pelo sistema operacional.

MÓDULO 2: O ECOSISTEMA

O que são Recursos Nativos?

Sensores Físicos: GPS, Acelerômetro, Giroscópio, Barômetro e Magnetômetro.

Captura & Mídia: Conjunto de Lentes, Sensores LiDAR e Microfones direcionais.

Segurança & Biometria: Enclave seguro com Face ID, Touch ID e BiometricPrompt.

Serviços do SO: Push Notifications, Bluetooth, Apple Pay e Google Wallet.

****enclave** é uma área isolada e altamente protegida*

Recurso Nativo x Cross-Platform

MÓDULO 2: O ECOSISTEMA

Deep Dive — Serviços de Localização

Múltiplas Fontes de Dados

O SO combina triangulação de antenas celulares, redes Wi-Fi próximas, Bluetooth e satélites (GPS/GLONASS/Galileo).

Hardware & Frequência

O módulo GPS dedicado é faminto por energia. Atualizações contínuas de alta precisão esgotam a bateria em poucos minutos se não forem otimizadas.

Recurso Nativo x Cross-Platform

MÓDULO 2: O ECOSISTEMA

SDK Nativo vs. Webview / Wrappers

Webview / Wrappers

Adequado para conteúdos estáticos ou formulários simples.
Sofre com gargalos graves de performance e fluidez ao tentar
exibir mapas ou renderizar navegação curva a curva.

SDK Nativo (Ex: Uber, iFood)

Indispensável para garantir 60/120 FPS, manipulação fluida de
vetores 3D e respostas imediatas aos gestos do usuário
durante a navegação.

Recurso Nativo x Cross-Platform

MÓDULO 2: O ECOSISTEMA

UI/UX — HIG vs. Material You

iOS (Human Interface Guidelines)

Foco em gestos das bordas, feedback tátil preciso via Taptic Engine, folhas modais do rodapé e navegação limpa.

Android (Material You)

Gestos globais de voltar, cores dinâmicas adaptadas ao papel de parede do usuário e botões de ação flutuantes (FAB).

Atenção: Forçar elementos gráficos do Android no iOS (ou vice-versa) gera forte rejeição e estranhamento pelos usuários.

MÓDULO 2: O ECOSISTEMA

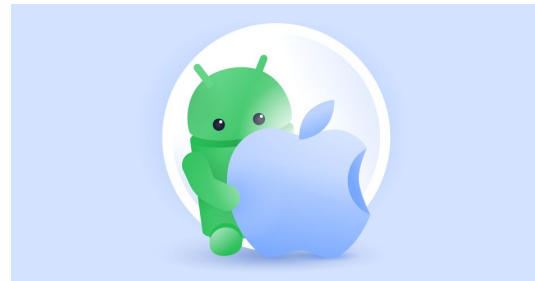
Performance de Renderização Gráfica

iOS: Metal API

SwiftUI e UIKit utilizam a biblioteca Metal para comunicação direta de baixíssima latência com a GPU da Apple, eliminando engargalos visuais.

Android: Vulkan / OpenGL

Jetpack Compose e View System utilizam drivers Vulkan otimizados, executando animações na thread dedicada de renderização.



A evolução da privacidade móvel: Como cada sistema operacional protege a integridade e os dados do usuário.



Arquivo Declarativo Mandatório: Todas as solicitações devem ser configuradas previamente no arquivo Info.plist.



NSLocationWhenInUseUsageDescription: Explicação clara para uso em primeiro plano.



NSLocationAlwaysAndWhenInUseUsageDescription: Explicação detalhada para uso em segundo plano.



Risco de Rejeição: Se a mensagem descritiva for genérica, a submissão será rejeitada pela equipe de revisão da App Store.

MÓDULO 3: PERMISSÕES



Precise Location

O usuário pode escolher se concede a localização exata por GPS ou um raio genérico aproximado (~1 a 10 km).



When In Use

O acesso é estritamente autorizado enquanto a interface gráfica do app estiver visível na tela.



Always Allow

Acesso liberado em segundo plano, exigindo confirmação periódica do usuário através de alertas do SO.

Permissões Android (Manifest)

MÓDULO 3: PERMISSÕES



Declaração Estática: Configurado dentro do arquivo AndroidManifest.xml.



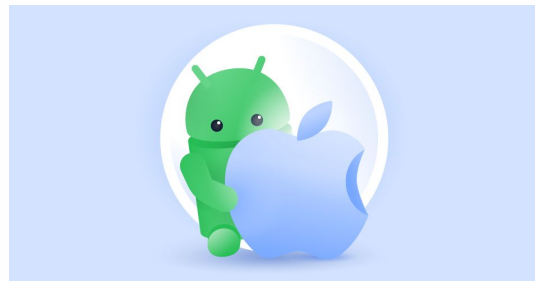
ACCESS_FINE_LOCATION: Permite leitura precisa do receptor de GPS.



ACCESS_COARSE_LOCATION: Permite estimativa aproximada baseada em Wi-Fi e antenas.



ACCESS_BACKGROUND_LOCATION: Acesso em segundo plano que exige aprovação técnica adicional no console do Google Play.



Níveis de Permissões Android

MÓDULO 3: PERMISSÕES



Runtime Permissions

Solicitadas dinamicamente ao usuário durante o uso a partir do Android 6.0 (API 23).



Apenas Desta Vez

Concessão temporária de uso único. A permissão é revogada imediatamente ao fechar o app.



Separação Obrigatória

Não é permitido solicitar acesso em background simultaneamente ao primeiro plano. Deve ser solicitado em etapas isoladas.

MÓDULO 3: PERMISSÕES



iOS: Pontos verde/laranja na barra de status indicam câmera e microfone ativos. Banner azul indica localização em background.



Android: Indicador verde no canto superior direito para sensores de mídia e ícone persistente para GPS.



Auditoria: Painel de Privacidade exibe o histórico exato de quando cada app acessou os recursos.

MÓDULO 3: PERMISSÕES

-  **Anti-Padrão:** Solicitar todas as permissões no primeiro segundo em que o usuário abre a aplicação.
-  **Padrão Recomendado:** Explicar o motivo e valor claro antes do alerta nativo do sistema. Ex: "Precisamos do GPS para mostrar os restaurantes ao seu redor."
-  **Tratamento de Negativas:** Se negado, o aplicativo deve continuar funcional, ocultando apenas o recurso estritamente dependente.

Mapas Nativos e Híbridos

MÓDULO 4: ARQUITETURA & CÓDIGO

Flutter (Platform Views)

Renderiza visualizações nativas dentro do widget tree usando `AndroidView` para o Google Maps e `UIKitView` para o Apple Maps.

React Native (Native Components)

Usa a biblioteca `react-native-maps`, que traduz os componentes JSX diretamente para o `MKMapView` (iOS) e `MapView` (Android).

Atenção: A acoplagem excessiva de views nativas na árvore híbrida consome mais memória e pode reduzir o FPS de scroll.

-  **GPS Desativado:** O usuário desabilitou o recurso globalmente nas configurações.
-  **Permissão Negada Permanentemente:** Marcado com "Não perguntar novamente".
-  **Modo de Economia Severo:** O SO reduz a amostragem de dados e limita a precisão.
-  **Solução Resiliente:** Redirecionar o usuário diretamente para as configurações do sistema através de Settings Intent ou App-Prefs.

MÓDULO 4: ARQUITETURA & CÓDIGO

- ✗ **Descrições Genéricas:** Textos vagos no Info.plist acarretam rejeição instantânea da Apple.
- ✗ **Permissões Não Utilizadas:** Declarar permissões no Manifesto que o app não consome ativamente.
- ✗ **Consumo Excessivo:** Aplicativos que drenam a bateria rapidamente em background são reprovados.
- ✓ Dominar as diretrizes nativas reduz drasticamente o tempo de revisão técnico nas lojas.

Apple (WWDC) e Google (Google I/O) lançam novas APIs anualmente (ex: Dynamic Island, Live Activities, widgets interativos).

Enquanto frameworks híbridos dependem do tempo de desenvolvimento da comunidade para criar bibliotecas, desenvolvedores nativos entregam os novos recursos no mesmo dia do lançamento oficial do sistema operacional.

Exercício

O Desafio

Implemente um serviço de localização em tempo no app híbrido que exiba na tela

Adicionar permissão

Teste: Expo

Exemplo

```
import { AppleMaps, GoogleMaps } from 'expo-maps';
```

⚙️ Dicas

```
npx create-expo-app@latest mapa-app
```

```
cd mapa-app
```

```
npm install
```

```
npx expo start
```

```
npx expo install expo-maps
```

```
npx expo install expo-dev-client
```

Permite criar uma
versão do app com
módulos nativos
adicionais

Depois Criar a API Key do Google Maps:

https://console.cloud.google.com/?utm_source=chatgpt.com

Exercício

Dicas

```
if (Platform.OS === 'android') {  
  return <GoogleMaps.View />;  
}
```

```
if (Platform.OS === 'ios') {  
  return <AppleMaps.View />;  
}
```


Exercício

Dicas

ios

O `expo-maps` usa:

Apple Maps



MapKit

Exercício

Dicas

Android

Precisamos configurar:

Google Cloud



Google Maps SDK for Android



API Key

Como fazer?

⚙️ Dicas

npx create-expo-app@latest mapa-app

cd mapa-app

npm install

npx expo install react-native-maps

npx expo start

npx expo install expo-location

☒ Google Cloud
☒ API Key
☒ expo-maps
☒ expo-dev-client

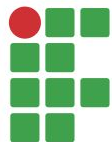
☒ Expo
☒ react-native-maps

```
import MapView from 'react-native-maps';
```

```
export default function App() {  
  return (  
    <MapView  
      style={{ flex: 1 }}  
      initialRegion={{  
        latitude: -20.4697,  
        longitude: -54.6201,  
        latitudeDelta: 0.0922,  
        longitudeDelta: 0.0421,  
      }}  
    />  
  );  
}
```

```
const { status } = await  
Location.requestForegroundPermissionsAsync();
```

milena.alegre@ifms.edu.br



INSTITUTO FEDERAL
Mato Grosso do Sul